Spark 运行机制理解

第五题：详解 Spark Driver 贴 Spark Intellij 源码截图加分

Spark Driver 详解：运行机制与源码剖析

一、Driver 的核心定位

在 Spark 架构中，**Driver 是应用程序的主控进程**，负责执行用户编写的 main 方法，并将代码转换为分布式任务分发到集群执行。可以这样理解：Driver 是 Spark 作业的"大脑"，而 Executor 是执行具体计算的"双手"。

Driver 进程在集群中的位置取决于部署模式：

> **Cluster 模式**：Driver 运行在集群的 ApplicationMaster 进程中
> **Client 模式**：Driver 运行在提交任务的客户端机器上

二、Driver 的核心职责

Driver 在 Spark 作业执行中承担以下关键职责：

职责	具体说明
程序转换	将用户编写的代码转化为 Job，并划分 Stage
任务调度	在 Executor 之间调度和分发 Task
状态跟踪	跟踪 Executor 的执行情况和任务进度
UI 展示	通过 Web UI（默认 4040 端口）展示运行状态

三、Driver 的核心组件

Driver 内部通过三个核心组件协同工作：

text 复制 下载

```
┌──────────────────────────────────────────────────┐
│                     Driver                       │
│  ┌──────────────────────────────────────────┐    │
│  │              SparkContext                 │    │
│  │  ┌──────────────┐  ┌──────────────┐      │    │
│  │  │ DAGScheduler  │  │ TaskScheduler │      │    │
│  │  │ (Stage级调度) │  │ (Task级调度)  │      │    │
│  │  └──────────────┘  └──────────────┘      │    │
│  │  ┌──────────────┐                         │    │
│  │  │SchedulerBackend│ (资源申请与通信)       │    │
│  │  └──────────────┘                         │    │
│  └──────────────────────────────────────────┘    │
└──────────────────────────────────────────────────┘
```

四、Driver 启动流程源码分析

以下源码分析基于 Standalone 集群模式

```scala
import ...

private[deploy] class Master(
    override val rpcEnv: RpcEnv,
    address: RpcAddress,
    webUiPort: Int,
    val securityMgr: SecurityManager,
    val conf: SparkConf)
  extends ThreadSafeRpcEndpoint with Logging with LeaderElectable {

  private val forwardMessageThread =
    ThreadUtils.newDaemonSingleThreadScheduledExecutor("master-forward-message-thread")

  // For application IDs
  private def createDateFormat = new SimpleDateFormat("yyyyMMddHHmmss", Locale.US)

  private val workerTimeoutMs = conf.get(WORKER_TIMEOUT) * 1000
  private val retainedApplications = conf.get(RETAINED_APPLICATIONS)
  private val retainedDrivers = conf.get(RETAINED_DRIVERS)
  private val reaperIterations = conf.get(REAPER_ITERATIONS)
```

4.1 第一步：Master 接收 Driver 提交请求

当客户端提交应用程序后，Master 收到 `RequestSubmitDriver` 消息，创建 DriverInfo 并加入等待队列

```scala
    case CheckForWorkerTimeOut =>
      timeOutDeadWorkers()

  }

  override def receiveAndReply(context: RpcCallContext): PartialFunction[Any, Unit] = {
    case RequestSubmitDriver(description) =>
      if (state != RecoveryState.ALIVE) {
        val msg = s"${Utils.BACKUP_STANDALONE_MASTER_PREFIX}: $state. " +
          "Can only accept driver submissions in ALIVE state."
        context.reply(SubmitDriverResponse(self, false, None, msg))
      } else {
        logInfo("Driver submitted " + description.command.mainClass)
        val driver = createDriver(description)
        persistenceEngine.addDriver(driver)
        waitingDrivers += driver
        drivers.add(driver)
        schedule()
```

4.2 第二步：Master 调度资源分配

Master 的 `schedule()` 方法负责为等待的 Driver 分配 Worker 节点

```scala
      } else {
        RecoveryState.RECOVERING
      }
      logInfo("I have been elected leader! New state: " + state)
      if (state == RecoveryState.RECOVERING) {
        beginRecovery(storedApps, storedDrivers, storedWorkers)
        recoveryCompletionTask = forwardMessageThread.schedule(new Runnable {
          override def run(): Unit = Utils.tryLogNonFatalError {
            self.send(CompleteRecovery)
          }
        }, workerTimeoutMs, TimeUnit.MILLISECONDS)
      }

    case CompleteRecovery => completeRecovery()

    case RevokedLeadership =>
      logError("Leadership has been revoked -- master shutting down.")
      System.exit(0)
```

4.3 第三步：Worker 启动 DriverRunner

Worker 收到 `LaunchDriver` 消息后，创建 DriverRunner 并启动

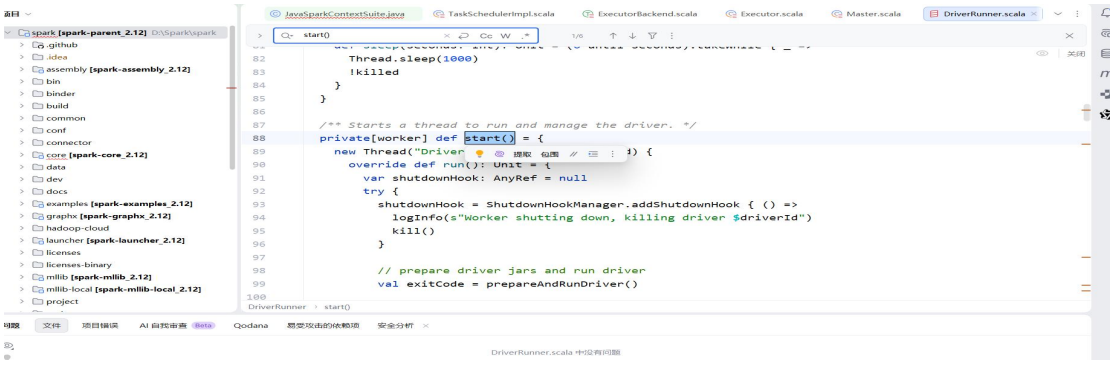
```scala
private[deploy] class DriverRunner(
    conf: SparkConf,
    val driverId: String,
    val workDir: File,
    val sparkHome: File,
    val driverDesc: DriverDescription,
    val worker: RpcEndpointRef,
    val workerUrl: String,
    val workerWebUiUrl: String,
    val securityManager: SecurityManager,
    val resources: Map[String, ResourceInformation] = Map.empty)
  extends Logging {

  @volatile private var process: Option[Process] = None
  @volatile private var killed = false

  // Populated once finished
  @volatile private[worker] var finalState: Option[DriverState] = None
  @volatile private[worker] var finalException: Option[Exception] = None
```

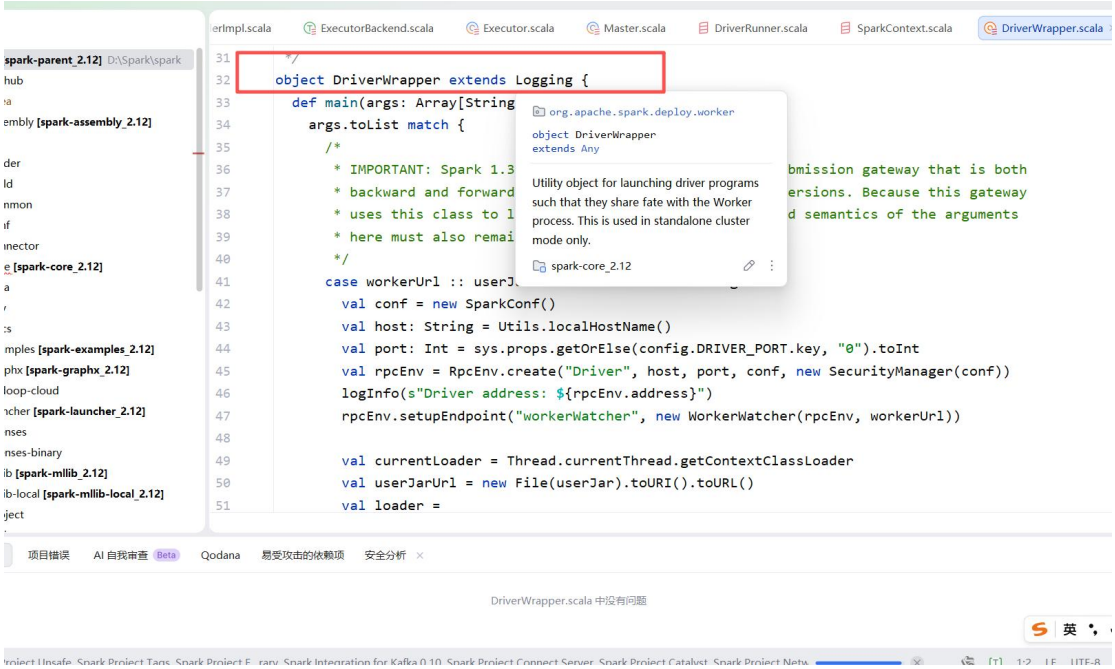
4.4 第四步：DriverRunner 执行 Driver

DriverRunner 的 `start()` 方法启动独立线程来运行 Driver：



4.5 第五步：DriverWrapper 执行用户代码

启动的 Driver 进程实际执行的是 `DriverWrapper` 的 main 方法，通过反射调用用户提交的 main 方法



五、Driver 完整启动流程图

```
text

Client ─提交应用─► Master                    │                    ▼                    创建 DriverInfo
                加入 waitingDrivers                    │                    ▼                    schedule() 调
度                    查找可用 Worker                    │                    ▼                    launchDriver(w
orker, driver)                    │                    ▼Worker ◄──────────┘    │    ▼创建 DriverRu
nner    │    ▼driver.start() ──► 创建新线程                    │                    ▼                    创建
工作目录                    下载 JAR 包                    构建启动命令                    │
▼                    launchDriver 执行                    │                    ▼                    DriverWrapper main
```

```
() 执行                    |                    ▼        反射调用用户 main 方法
|                    ▼          new SparkContext()
```

七、总结

Driver 是 Spark 应用程序的核心协调者，其运行机制可以概括为：

> **角色定位**：应用程序的主控进程，执行 main 方法
> **核心职责**：程序转换、任务调度、状态跟踪、UI 展示
> **内部组件**：SparkContext、DAGScheduler、TaskScheduler、SchedulerBackend
> **启动流程**：Master 接收 → 调度 Worker → Worker 启动 DriverRunner → 执行用户代码

理解 Driver 的运行机制，对于 Spark 任务调优和问题排查至关重要。